



AURALIS NATURA
HOLISTIC HEALTH

TECHNISCHER BAU- & LAUNCH-LEITFADEN

Die Kunden-App *bauen & veröffentlichen.*

Der komplette Leitfaden — für absolute Einsteiger. Was, wo, wann und wie — mit jedem Link, den du brauchst. Gebaut für maximale Skalierbarkeit, mühelose Updates und volle DSGVO-Konformität. Recherchiert nach dem Stand der Technik 2026.

STAND JULI 2026

VERTRAULICH

KEIN VORWISSEN NÖTIG — GLOSSAR IN KAPITEL 1

SO Liest DU DIESEN LEITFADEN

Dieses Dokument erklärt alles, was nötig ist, um aus deinem bestehenden Kunden-Portal eine echte App für Apple App Store und Google Play zu machen — Schritt für Schritt, ohne dass du programmieren können musst. Jeder Bau-Schritt ist markiert: **CLAUDE** = mache ich für dich · **DU** = dein aktiver Teil (meist 5–15 Minuten). Fachbegriffe stehen erklärt in Kapitel 1, ganz hinten eine Link-Sammlung.

WAS DU AM ENDE HAST

Eine App im gleichen Auralis-Look wie dein Portal, installierbar aus beiden Stores, mit Push-Nachrichten („Dein Bericht ist bereit“), Face-ID-Login, Offline-Zugang und der Möglichkeit, Programme direkt in der App zu kaufen (via Stripe, ohne Apple-Provision). Updates spielst du danach ohne erneute Store-Prüfung ein.

- | | | | |
|----|---|----|---------------------------------|
| 01 | Glossar — jeder Begriff einfach erklärt | 05 | Skalieren & mühelose Updates |
| 02 | Das große Bild & unsere Entscheidungen | 06 | DSGVO & Gesundheitsdaten |
| 03 | Die Konten, die du anlegst | 07 | Kauf direkt in der App (Stripe) |
| 04 | Schritt für Schritt: die App bauen | 08 | Wartung & der Jahresrhythmus |
| | | 09 | Kosten auf einen Blick |

01 Glossar — jeder Begriff einfach erklärt

Lies das einmal quer; du musst nichts auswendig lernen. Immer wenn im Text ein Begriff auftaucht, kannst du hier nachschlagen.

Web-App / Web-Oberfläche

Deine bestehenden Seiten (Portal, Buchung) — gebaut aus HTML/CSS/JavaScript, also der Sprache von Webseiten. Genau das steckt schon fertig in deinem Projekt.

Native App

Eine „echte“ App, die man aus dem Store lädt und die ein Icon auf dem Homescreen hat.

Capacitor

Das Werkzeug, das deine Web-App in eine native App verwandelt — für iPhone und Android aus einer Vorlage. Es legt eine dünne „native Hülle“ um deine Web-Oberfläche und gibt ihr Zugriff auf Handy-Funktionen (Kamera, Face ID, Push).

WebView

Das Fenster in der App, das deine Web-Oberfläche anzeigt. Für die Kundin unsichtbar — sie sieht nur „die App“.

Gebündelt (bundled)

Die Web-Dateien liegen in der App auf dem Handy (nicht auf einem Server). Vorteil: die App startet sofort und funktioniert offline. Das ist der empfohlene Weg.

OTA-Update / Live-Update

„Over the air“ = über die Luft. Ein Update deiner Inhalte (Texte, Design, Abläufe), das direkt aufs Handy kommt — ohne dass die Kundin etwas aus dem Store neu laden muss und ohne erneute Apple/Google-Prüfung. Das macht Updates mühelos. Wir nutzen dafür den Dienst Capgo.

App Store Connect

Apples Web-Seite, auf der du deine iPhone-App verwaltest und einreichst.

Play Console

Das Gegenstück bei Google für Android.

TestFlight

Apples Test-App: damit installierst du deine App vorab auf dein eigenes iPhone, bevor sie öffentlich ist. Keine Store-Prüfung nötig.

Xcode

Apples kostenloses Bau-Programm, läuft nur auf einem Mac (den hast du). Damit wird die iPhone-App erzeugt und hochgeladen.

Android Studio

Das Gegenstück für Android; läuft auf Mac/Windows.

Signieren / Provisioning

Ein digitaler „Ausweis“, der beweist, dass die App wirklich von dir kommt. Xcode kann das automatisch — ein Klick.

SDK

„Software Development Kit“ — der Werkzeugkasten einer Plattform. Apple/Google verlangen regelmäßig, dass Apps mit einer aktuellen Version gebaut werden (Grund für gelegentliche Neubauten, siehe Wartung).

Firebase / FCM

Googles kostenloser Dienst, der Push-Nachrichten an alle Handys verschickt („Firebase Cloud Messaging“). Er ist die Zentrale für „Dein Bericht ist bereit“.

APNs

„Apple Push Notification service“ — Apples Kanal für Push aufs iPhone. Man erzeugt dafür einen kleinen Schlüssel (eine .p8-Datei) im Apple-Konto.

Stripe Payment Sheet

Das schicke Bezahlfenster von Stripe, das sich in der App öffnet — inklusive Apple Pay und Google Pay in einem Fingertipp.

IAP (In-App-Purchase)

Apples/Googles eigenes Bezahlungssystem, bei dem sie 15–30 % Provision nehmen. Gilt nur für digitale Güter — deine Coachings sind Dienstleistungen und dürfen Stripe nutzen (Kapitel 7).

VPS / Server

Ein kleiner Computer im Rechenzentrum, der rund um die Uhr läuft. Dorthin ziehen wir später den Portal-Server, damit er nicht von Desirees Mac abhängt.

Docker / Coolify

Docker packt deine App in eine „Kiste“, die überall gleich läuft. Coolify ist eine einfache Oberfläche, die diese Kiste auf dem Server startet, SSL einrichtet und bei jedem Git-Push automatisch neu ausrollt — quasi „Vercel für deinen eigenen Server“.

DSGVO Art. 9 / Art. 32

Gesundheitsdaten sind besonders geschützt (Art. 9 → ausdrückliche Einwilligung nötig). Art. 32 verlangt technische Schutzmaßnahmen wie Verschlüsselung (hast du bereits: Fernet).

Wir bauen nicht neu. Deine Portal-Oberfläche existiert schon und sieht premium aus. Capacitor legt die native Hülle darum — eine Codebasis, beide Stores. Vier Entscheidungen prägen das Projekt; jede folgt der 2026er-Best-Practice:

1 Gebündelt statt „Server-URL“

Die Web-Dateien liegen in der App (gebündelt). So startet sie sofort und funktioniert offline. Der Alternativweg („die App lädt beim Start alles von einem Server“) gilt laut Capacitor-Doku ausdrücklich nicht für den Produktivbetrieb — bei schlechtem Netz sähe die Kundin einen leeren Bildschirm.

Quelle: capacitorjs.com/docs/guides/deploying-updates

2 Mühelose Updates mit Capgo (OTA)

Inhalts-Updates (Texte, Design, Abläufe) schieben wir direkt aufs Handy — ohne erneute Store-Prüfung. Apple und Google erlauben das ausdrücklich für Web-Inhalte (JavaScript/CSS/Design); nur echte native Änderungen gehen durch die Store-Prüfung. Werkzeug der Wahl ist Capgo: Apples haus eigene Alternative „Appflow“ wird eingestellt (keine Neukunden mehr, Ende 2027 Abschaltung). Capgo ist günstig (\$12/Monat), verschlüsselt und aktiv gepflegt.

Quellen: capgo.app · OTA vs. App-Store-Regeln

3 Der Server zieht auf einen EU-Rechner

Für eine echte App, auf die sich Kundinnen verlassen, sollte der Portal-Server immer laufen — nicht nur, wenn Desirees Mac wach ist. Empfehlung mit dem besten Preis-Leistungs-Verhältnis in der EU: ein kleiner Hetzner-Server (ab ~€4/Monat, deutsche Rechenzentren → DSGVO-freundlich) mit Coolify für automatische Updates & SSL. Wer es ganz einfach mag, nimmt Render (verwaltet, EU-Region, ~\$7/Monat). Beide sind für Gesundheitsdaten besser als US-Anbieter.

Quellen: hetzner.com/cloud · coolify.io · render.com

4 Bezahlen mit Stripe — ohne Apple-Provision

Deine Programme sind persönliche Dienstleistungen (1-zu-1-Coaching). Apples Regel 3.1.3(d) erlaubt für solche „Person-zu-Person“-Dienste ausdrücklich eigene Bezahlwege — also Stripe, ohne die 15–30 % Provision. Du behältst ~98,5 % (nur Stripe-Gebühr). Details & die eine Grenze in Kapitel 7.

Quellen: App-Store-Richtlinien 3.1.3 · Stripe Payment Sheet

03 Die Konten, die du anlegst

Diese Konten brauchst du. Einige hast du schon. Lege die drei mit Wartezeit (Apple, Google, Firebase) zuerst an — der Rest geht sofort.

KONTO	WOFÜR	ADRESSE	KOSTEN	DEIN AUFWAND
Apple Developer	iPhone-App veröffentlichen	developer.apple.com/programs	\$99/Jahr	~20 Min · Freigabe 24–48 h
Google Play Console	Android-App veröffentlichen	play.google.com/console	\$25 einmalig	~15 Min
Firebase	Push-Nachrichten	console.firebase.google.com	kostenlos	~5 Min
Cappgo	Mühelose OTA-Updates	cappgo.app	~\$12/Monat	~5 Min
Stripe	Bezahlen in der App	dashboard.stripe.com	pro Transaktion	hast du schon
Hetzner oder Render	Server (Portal-Backend)	hetzner.com / render.com	~€4–7/Monat	~10 Min · optional/später
Apple ID / Google-Konto	Login-Basis für obiges	—	kostenlos	hast du schon

WICHTIGE WEICHE BEI APPLE

Einzelperson (Individual) ist schneller freigeschaltet und reicht für den Start. Ein Firmen-Konto zeigt den Firmennamen im Store, braucht aber eine D-U-N-S-Nummer (kostenlos, kann aber 1–2 Tage dauern). Für morgen/schnell: Einzelperson. Apple lässt den Typ später nicht einfach wechseln — sag mir vorher Bescheid, welchen du willst.

04 Schritt für Schritt: die App bauen

In Reihenfolge. Bei jedem Schritt steht, wer ihn macht und wie lange dein Teil dauert.

A Web-Oberfläche app-fertig machen

CLAUDE

Ich richte eine kleine, für die App optimierte Fassung deines Portals ein (gleicher Look, plus native Navigation). Dein Aufwand: 0 · Werkzeug: das bestehende Repo.

B Capacitor-Projekt anlegen & Plattformen hinzufügen

CLAUDE

Ich installiere Capacitor und erzeuge die iOS- und Android-Projekte (npm i @capacitor/core, npx cap add ios, npx cap add android). Dein Aufwand: 0 · Anleitung: capacitorjs.com/docs/getting-started

C App-Icon & Startbildschirm aus dem Siegel

CLAUDE

Aus deinem botanischen Siegel erzeuge ich alle Icon- & Splash-Größen automatisch mit @capacitor/assets (ein Logo rein → alle Formate raus). Dein Aufwand: 0 · Werkzeug: capacitor-assets

D Native Funktionen: Push, Face ID, Offline, Stripe

CLAUDE

DU

Ich baue Push (Plugin @capacitor-firebase/messaging), Face-ID-Login, Offline-Speicher und das Stripe-Bezahlfenster ein. Dein Teil: im Apple-Konto einen APNs-Schlüssel (.p8) erzeugen und in Firebase hochladen (ich zeige jeden Klick); Firebase-Projekt benennen. Dein Aufwand: ~10 Min · Hinweis: Push testet man nur auf einem echten iPhone, nicht im Simulator.

E Mühelose Updates verdrahten (Capgo)

CLAUDE

DU

Ich verbinde die App mit Capgo, damit du künftig Inhalts-Updates per Knopfdruck ausspielst. Dein Teil: Capgo-Konto anlegen, einen Schlüssel kopieren. Dein Aufwand: ~5 Min · Anleitung: capgo.app

F iPhone-App bauen & auf dein Handy (TestFlight)

DU

CLAUDE

Ich bereite alles in Xcode vor. Dein Teil (an Desirees Mac, ich führe dich Klick für Klick): in Xcode mit deiner Apple-ID anmelden, „Automatisch signieren“ aktivieren, auf „Run“ drücken → die App landet auf dem iPhone. Danach Test über TestFlight. Dein Aufwand: ~20 Min

G Android-App bauen

CLAUDE

Ich erzeuge das Android-Paket (AAB) in Android Studio und die Store-Screenshots aus deinen echten Bildschirmen. Dein Aufwand: 0

H Store-Einträge, Datenschutz & „Data Safety“

CLAUDE

DU

Ich entwerfe alles: Beschreibung (DE/EN/ES), Keywords, Datenschutz-Text, Apples Privacy-Labels und Googles Data-Safety-Formular (14 Datenkategorien) inkl. Googles Health-Apps-Erklärung. Dein Teil: Texte kurz prüfen, Antworten bestätigen, „Einreichen“ klicken. Dein Aufwand: ~30 Min

Ich packe den Flask-Server in Docker und richte ihn mit Coolify auf Hetzner ein (oder auf Render) — inkl. HTTPS & Auto-Deploy aus GitHub. Dein Teil: Hetzner-/Render-Konto anlegen, Zahlungsmittel hinterlegen. Dein Aufwand: ~10 Min

05 Skalieren & mühelose Updates

So läuft ein Update, wenn die App live ist

- **Inhalt ändern (Text, Preis, Design, Ablauf):** Ich baue neu, spiele es über Capgo aus → deine Kundinnen haben es beim nächsten App-Start, ohne Store-Prüfung, ohne Neuinstallation. Minuten statt Tage.
- **Native Änderung (neue Handy-Funktion, SDK-Pflicht):** geht einmalig durch die normale Store-Prüfung (24–48 h). Das ist selten — meist reicht Capgo.

So skaliert das System mit

- **Wenige bis Hunderte Kundinnen:** Der kleine Hetzner-/Render-Server genügt locker. Verschlüsselte SQLite reicht für den Start.
- **Wachstum:** Server-Größe per Klick erhöhen (mehr RAM/CPU). Bei Bedarf später auf eine EU-gehostete Postgres-Datenbank wechseln — gleicher Code-Ansatz.
- **Bilder/PDFs:** Über den Server oder einen EU-Objektspeicher ausliefern; die App selbst bleibt schlank (gebündelt).
- **Push in Masse:** Firebase FCM ist kostenlos bis in die Millionen — kein Engpass.

KERNIDEE

Eine Codebasis → beide Stores. Inhalte fließen per Capgo, Daten per API vom EU-Server. Du wächst, ohne die Architektur zu wechseln.

06 DSGVO & Gesundheitsdaten

Gesundheitsdaten sind „besondere Kategorien“ (Art. 9 DSGVO). Vieles hast du bereits richtig — hier die App-spezifischen Punkte:

- **Ausdrückliche Einwilligung (Art. 9):** ein eigener Zustimmungsschritt, der die Gesundheitsdaten benennt — nicht in AGB versteckt. Deine Buchung/Intake hat das bereits; in der App übernehmen wir es 1:1.

- **Verschlüsselung (Art. 32):** erledigt — deine Daten liegen Fernet-verschlüsselt. Auf dem Handy speichert die App Zugangs-Token im sicheren Schlüsselbund.
- **EU-Hosting:** nicht zwingend, aber der einfachste & sicherste Weg für Gesundheitsdaten. Deshalb Hetzner (DE) oder Render-EU.
- **Auftragsverarbeitung (AVV/DPA):** mit jedem Dienstleister abschließen, der Daten berührt — Hetzner/Render (Hosting), ggf. Firebase (nur Push-Token, keine Gesundheitsdaten!). Wir halten Gesundheitsdaten bewusst von Firebase fern.
- **Einwilligungs-Nachweise:** Zeitpunkt, Version des Textes, Umfang speichern — hast du im Kundinnen-Datensatz bereits angelegt.
- **Store-Formulare:** Apples Privacy-Labels & Googles Data-Safety müssen zur tatsächlichen App passen — Google prüft das automatisch gegen. Ich fülle sie wahrheitsgemäß aus.

LEITPLANKE BLEIBT

Coaching & Bildung, nicht Medizin. Die App-Beschreibung im Store muss das genauso klar sagen wie die Website — Apple prüft Gesundheits-Claims streng.

07 Kauf direkt in der App (Stripe)

Ja — und für dich ohne Apple/Google-Provision, weil Coaching eine Dienstleistung ist.

WAS VERKAUFT WIRD	BEZAHLWEG	PROVISION
Persönliche Dienstleistung (Coaching, Gespräch, Bericht)	Dein Stripe ✓	0 % (nur Stripe-Gebühr ~1,5 % + €0,25)
Rein digitales Selbstbedienungs-Produkt (E-Book, Videokurs)	Apple/Google-System	15-30 %

In der App: ein „Programme“-Screen (Root/Bloom/Flourishing/Grove) → antippen → Stripe-Bezahlfenster mit Apple Pay/Google Pay in einem Tipp → Zahlung landet im gleichen Stripe + in der Betriebskonsole („Bezahlt“), die Journey rückt automatisch weiter. Kein neues Bezahlssystem nötig.

DIE EINE GRENZE

Solange jedes bezahlte Angebot an die Coaching-/Bericht-Leistung gekoppelt ist, bleibst du im Stripe-Weg. Ein reines Download-Produkt ohne Mensch würde die Store-Provision auslösen.

Aufwand: ~½ Tag zusätzlich, da Stripe & „Bezahlt“-Logik schon existieren. Quellen: Stripe In-App-Zahlungen · Richtlinie 3.1.3(d)

08 Wartung & der Jahresrhythmus

Website, Portal und Konsole laufen praktisch wartungsfrei. Die App hat den einzigen „Herzschlag“:

AUFGABE	WIE OFT	DEIN AUFWAND (ICH MACHE DIE TECHNIK)
App gegen neues SDK neu bauen (Apple/Google-Pflicht)	1-2×/Jahr	~15 Min pro Einreichung
Verteil-Zertifikat erneuern	jährlich	~15 Min
Capgo-/Abhängigkeits-Updates	gelegentlich	~0 (mache ich)
Server-Sicherheits-Patches	2-3×/Jahr	~0 (mache ich)

SUMME

Mit meiner Hilfe ~2-4 Stunden pro Jahr, in kleinen Häppchen. Ohne Hilfe eher 1-3 Tage (fast alles davon der Store-Zyklus). Faustregel: Website/Portal/Konsole = einmal bauen, läuft. Die App ist das einzige Teil, das man gelegentlich pflegt — und diese Pflege kann ich übernehmen.

09 Kosten auf einen Blick

Einmalig (Start)

POSTEN	KOSTEN
Google Play Konto (lebenslang)	~€23
App bauen (ich mache es)	€0
Start-Summe	≈ €23

Pro Jahr (Betrieb)

POSTEN	PRO JAHR	NÖTIG?
Apple Developer	~€92	nur für iOS
Capgo (mühelose Updates)	~€135 (\$12/Mon.)	empfohlen
EU-Server (Hetzner/Render)	~€50-90	empfohlen für App-Betrieb
Domain	~€12	hast du
Claude (KI-Berichte)	~€216	hast du evtl. schon
Firebase Push · Cloudflare	€0	Gratis-Stufe

Realistisch neu dazu für die App: Apple €92 + Capgo €135 + Server ~€70 ≈ **€300/Jahr**.
 Ohne iOS (nur Android+PWA) und ohne Capgo entsprechend weniger. Stripe kostet nur pro Verkauf (~1,5 % + €0,25).

10 Wer macht was — deine aktiven Minuten

DEIN TEIL (AKTIV)	MINUTEN
Apple-Developer-Konto starten	~20
Google-Play-Konto starten	~15
Firebase-Projekt anlegen	~5
APNs-Schlüssel erzeugen	~5
Capgo-Konto	~5
Xcode: anmelden, signieren, „Run“	~10
App auf dem iPhone testen	~10
Store-Texte prüfen + Data-Safety bestätigen	~25
„Einreichen“ klicken (×2)	~5
Server-Konto (optional)	~10
Summe	≈ 100 Min

Alles andere — Programmierung, Konfiguration, Icons, Screenshots, sämtliche Texte, Datenschutz, Server-Einrichtung — mache ich.

11 Der 3-Tage-Launch-Plan

Voraussetzung: Apple-Konto am Morgen von Tag 1 starten (24–48 h Freigabe ist der einzige echte Engpass).

TAG	DU (AKTIV)	ICH (IM HINTERGRUND)
1	Konten starten: Apple, Google, Firebase (~40 Min)	Capacitor-Projekt, Icons/Splash, Android-Build, Store-Texte
2	APNs-Schlüssel · Xcode signieren · auf iPhone testen (~25 Min)	Push, Face ID, Offline, Stripe, Capgo verdrahten
3	Texte prüfen · Data-Safety bestätigen · einreichen (~30 Min)	Feinschliff, beide Stores einreichen

„Fertig in 3 Tagen“ heißt: beide Apps laufen auf deinem iPhone und sind eingereicht. Google ist meist Tag 3 live; Apple genehmigt üblicherweise 24–48 h nach Einreichung (also Tag 4–5).

12 Anhang: alle Links

Werkzeuge & Doku

- Capacitor — Start: capacitorjs.com/docs/getting-started · Updates/Deploy: [/guides/deploying-updates](https://capacitorjs.com/docs/guides/deploying-updates)
- Icons & Splash: github.com/ionic-team/capacitor-assets
- Capgo (OTA-Updates): capgo.app
- Push mit Firebase: capacitorjs.com/docs/guides/push-notifications-firebase
- Stripe In-App: docs.stripe.com/payments/mobile · Plugin: [Cap-go/capacitor-stripe-pay](https://capgo/capacitor-stripe-pay)

Konten & Stores

- Apple Developer: developer.apple.com/programs · Einreichen: [/app-store/submitting](https://developer.apple.com/app-store/submitting) · Richtlinien: [/review/guidelines](https://developer.apple.com/review/guidelines)
- Google Play Console: play.google.com/console · Data Safety: Hilfe 10787469 · Health: Hilfe 16679511
- Firebase: console.firebase.google.com · Stripe: dashboard.stripe.com

Server & Recht

- Hetzner Cloud: hetzner.com/cloud · Coolify: coolify.io · Render: render.com
- DSGVO: Art. 9 · Art. 32

